This debugging session actually taught several important ROS 2 concepts. Here's a summary you can keep as your notes.

Lesson 1: A ROS 2 node can have multiple publishers

Your Arducam node publishes:

```
/point_cloud     (PointCloud2)
/depth_frame     (Float32MultiArray)
/parameter_events
/rosout
```

A single node can publish as many topics as it needs.

Lesson 2: `ros2 node info` vs `ros2 topic info`

Use:

```
ros2 node info /arducam
```

to answer:

> **What does this node publish and subscribe to?**

Use:

```
ros2 topic info /point_cloud
```

to answer:

> **Who is publishing this topic and who is subscribing to it?**

Lesson 3: A publisher does NOT need a subscriber

Your Arducam continuously published `/point_cloud` even when nobody was listening.

```
Arducam
   |
```

▼
`/point_cloud`

Subscriber count:

`0`

This is completely normal.

Lesson 4: Subscriber count changes dynamically

Before launching the converter:

```
Publisher count: 1
Subscription count: 0
```

After launching `pointcloud_to_laserscan`:

```
Publisher count: 1
Subscription count: 1
```

ROS 2 updates these numbers in real time.

Lesson 5: `rqt_graph` only shows active connections

A topic is usually shown only when:

- there is a publisher
- there is a subscriber

No subscriber means the topic may disappear from the graph.

Lesson 6: `ros2 topic echo` is itself a subscriber

When you ran:

```
ros2 topic echo /point_cloud
```

you unknowingly created another subscriber.

So the graph changed because **you** subscribed.

Lesson 7: PointCloud2 was already correct

You verified:

```
ros2 topic echo /point_cloud --once
```

It contained

```
header
frame_id
x
y
z
```

which meant the Arducam node was publishing a valid 3D point cloud.

Lesson 8: LaserScan conversion works

Pipeline:

```
ToF Camera
      │
      ▼
PointCloud2
      │
      ▼
pointcloud_to_laserscan
      │
      ▼
LaserScan
```

You successfully reached this stage.

Lesson 9: What LaserScan actually is

LaserScan is **not another point cloud**.

It is simply:

```
Angle  → Distance
```

Example:

```
-90° → 1.8 m
-89° → 1.8 m
```

```
...
0°   → 0.7 m
...
90°  → 2.2 m
```

It is basically an array of

`(angle, range)`

This makes obstacle avoidance much easier.

Lesson 10: `ranges[]` is the important field

When you saw:

```
ranges:
- 0.12
- 0.18
- 0.25
...
```

Those numbers were distances in **meters**.

For example,

`0.25`

means

`Obstacle is 25 cm away`

at that particular angle.

Lesson 11: SSH was NOT the problem

SSH only gives you a remote terminal.

It doesn't stop ROS nodes from communicating.

Lesson 12: TF (`/tf_static`) was missing

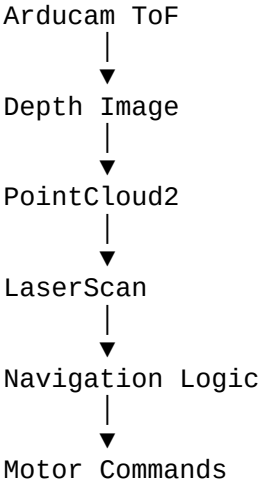
Your robot currently has no TF tree.

`ros2 topic echo /tf_static`

returned nothing.

Although we still got the converter working, you'll eventually want a proper TF tree when adding mapping or visualization.

Final architecture achieved



The biggest takeaway

You've crossed an important milestone. Before this, the camera was just producing raw sensor data. Now you've converted it into a **LaserScan**, which is the standard input used by many robotics navigation and obstacle avoidance algorithms.

The next step is to write a node that reads `/scan`, identifies free and blocked directions, and decides how your robot should move. That's where your robot starts making autonomous navigation decisions.